Name: Lucky Sharma

PRN: 202301040253

IA Activity : 3

Jenkins CI/CD + Terraform Mini Project

❖ TECHNICAL REPORT

Implementation of CI/CD Pipeline using Jenkins, Docker, Terraform and AWS

Subject: Cloud and Network Deployment (CND)
Project Title: Automated CI/CD Pipeline for an E-Commerce Web Application using Jenkins, Docker and Terraform
Platform: Amazon Web Services (AWS)
Operating System: Ubuntu 24.04 LTS
Application: E-Commerce Web Application

1. Abstract

This project focuses on the implementation of an automated Continuous Integration and Continuous Deployment (CI/CD) pipeline for an e-commerce web application using Jenkins, Docker, Terraform and Amazon Web Services (AWS).

The primary objective is to automate the application development and deployment process, reducing manual intervention and improving deployment consistency.

Terraform is used to provision the required AWS infrastructure, including an EC2 virtual machine and its associated security group. Jenkins is installed on the EC2 instance and configured to integrate with a GitHub repository. Docker is used to containerize the application and provide a consistent runtime environment.

The Jenkins pipeline consists of four major stages: Checkout, Build, Test and Deploy. Jenkins periodically monitors the GitHub repository using Poll SCM and automatically triggers the pipeline when new code changes are detected.

The implementation was successfully tested through both manual and automatic pipeline execution. The application was built, tested and deployed on the AWS EC2 instance and accessed through its public IP address.

2. Introduction

In traditional software deployment processes, developers often perform several tasks manually, including transferring source code, configuring environments, building applications, testing and deploying them on servers.

These manual processes can be time-consuming and may introduce errors or inconsistencies.

Continuous Integration and Continuous Deployment provide an automated approach to software development and delivery.

In this project, we implement a CI/CD workflow by integrating multiple DevOps technologies:

- GitHub for source code management.
- Jenkins for continuous integration and deployment automation.

- Docker for application containerization.
- Terraform for Infrastructure as Code (IaC).
- AWS EC2 for cloud-based application hosting.

The integration of these technologies provides an automated deployment workflow in which code changes pushed to GitHub can be detected, built, tested and deployed automatically.

3. Objectives

The main objectives of this project are:

1. To understand and implement the concepts of Continuous Integration and Continuous Deployment.
2. To provision cloud infrastructure using Terraform.
3. To configure and manage an AWS EC2 instance.
4. To install and configure Jenkins as a CI/CD automation server.
5. To integrate Jenkins with GitHub for source code management.
6. To containerize an e-commerce application using Docker.
7. To implement automated Checkout, Build, Test and Deploy stages.
8. To configure automatic pipeline triggering using Jenkins Poll SCM.
9. To deploy and access the application through an AWS EC2 public IP address.

4. Technologies and Tools Used

Technology	Purpose
AWS EC2	Provides the virtual server for hosting Jenkins and the application
Terraform	Automates cloud infrastructure provisioning
Jenkins	Automates the CI/CD pipeline
Docker	Builds and runs application containers
Git	Tracks source code changes
GitHub	Hosts the project repository
Ubuntu 24.04 LTS	Server operating system
HTML/CSS	Frontend application technologies
VS Code	Development and configuration environment

5. System Architecture

The project follows an integrated CI/CD architecture in which GitHub acts as the source code repository, Jenkins manages automation, Docker handles containerization and AWS EC2 provides the hosting environment.

5.1 Architecture Workflow

Developer → GitHub → Jenkins → Docker Build → Application Testing → Deployment → AWS EC2

5.2 Working Description

1. The developer makes changes to the application source code.
2. Updated source code is pushed to the GitHub repository.
3. Jenkins periodically checks the repository using Poll SCM.
4. When a new commit is detected, Jenkins automatically triggers the pipeline.
5. Jenkins checks out the latest source code.
6. Docker builds an image using the Dockerfile.
7. Jenkins starts a temporary container and tests the application.
8. If testing succeeds, Jenkins deploys the application container.
9. The deployed application becomes accessible through the EC2 public IP address.

6. Infrastructure Provisioning using Terraform

Terraform was used to implement Infrastructure as Code (IaC).

Instead of manually creating AWS resources through the management console, infrastructure requirements were defined using Terraform configuration files.

6.1 Terraform Configuration Files

The Terraform directory contains the following files:

main.tf

Defines the AWS provider, Ubuntu AMI data source, security group and EC2 instance resource.

variables.tf

Defines configurable parameters such as:

- AWS region
- EC2 instance type
- Existing EC2 key pair name
- Allowed SSH CIDR address

outputs.tf

Defines useful output values, including:

- EC2 public IP address
- Jenkins dashboard URL
- Deployed website URL

terraform.tfvars

Contains environment-specific variable values. This file is maintained locally and excluded from the GitHub repository to avoid exposing environment-specific information.

6.2 AWS Infrastructure Configuration

The following infrastructure was provisioned:

Configuration	Value
Cloud Provider	AWS
Region	Asia Pacific (Mumbai)
Operating System	Ubuntu 24.04 LTS
Instance Type	t3.micro
Root Storage	20 GB gp3
Security Group	Activity3-Jenkins-SG
EC2 Instance	Activity3-Jenkins-Server

6.3 Security Group Configuration

The security group controls incoming network traffic to the EC2 instance.

Port	Protocol	Purpose
22	TCP	SSH access
8080	TCP	Jenkins dashboard
80	TCP	E-commerce website

SSH and Jenkins dashboard access were restricted to the configured public IP address, while HTTP access was allowed publicly to make the website accessible.

6.4 Terraform Execution

The following commands were used:

terraform init

Initializes Terraform and downloads the required AWS provider.

terraform fmt

Formats Terraform configuration files.

terraform validate

Checks the validity of Terraform configurations.

terraform plan

Previews the infrastructure changes before applying them.

terraform apply

Creates the specified AWS infrastructure.

7. Jenkins Installation and Configuration

Jenkins was installed on the Ubuntu EC2 instance and configured as the CI/CD automation server.

7.1 Java Installation

Jenkins requires Java to execute.

Java 21 was installed using:

sudo apt update

sudo apt install -y fontconfig openjdk-21-jre

The Java installation was verified using:

java -version

7.2 Jenkins Installation

The Jenkins package repository was configured and Jenkins was installed using the official package repository.

The Jenkins service was enabled and started:

sudo systemctl enable jenkins

sudo systemctl start jenkins

Service status was checked using:

sudo systemctl status jenkins

The Jenkins dashboard was accessed through:

http://<EC2-PUBLIC-IP>:8080

The initial administrator password was retrieved from the Jenkins server, and the initial setup was completed by installing the suggested plugins and creating an administrator account.

7.3 Jenkins Job Configuration

A Pipeline job named Ecommerce-CI-CD was created.

The following configuration was used:

Parameter	Configuration
Job Name	Ecommerce-CI-CD
Pipeline Definition	Pipeline script from SCM
SCM	Git
Repository	CND_Activity_3
Branch	main
Script Path	Jenkinsfile
Credentials	None (public repository)

Jenkins was connected directly to GitHub to retrieve the Jenkinsfile and application source code.

8. Docker Installation and Configuration

Docker was used to package the e-commerce application into a container.

8.1 Docker Installation

Docker was installed on the Ubuntu EC2 instance:

sudo apt update

sudo apt install -y docker.io

The Docker service was enabled and started:

sudo systemctl enable docker

sudo systemctl start docker

Installation was verified using:

docker --version

8.2 Jenkins-Docker Integration

To allow Jenkins to execute Docker commands, the Jenkins user was added to the Docker group.

sudo usermod -aG docker jenkins

sudo systemctl restart jenkins

Docker access was verified using:

sudo -u jenkins docker ps

This allowed Jenkins pipeline stages to build and manage Docker containers.

8.3 Dockerfile

The Dockerfile defines the instructions required to build the application image.

It packages the application files and configures the container to serve the frontend application.

Docker images are generated using:

docker build -t ecommerce-app .

Containers are created from these images to execute the application.

9. Jenkins CI/CD Pipeline Implementation

The CI/CD pipeline was implemented using a declarative Jenkinsfile.

The pipeline consists of four major stages.

9.1 Checkout Stage

The Checkout stage retrieves the source code from the configured GitHub repository.

```
stage('Checkout') {
    steps {
        echo 'Checking out source code...'
```

```
        checkout scm
    }
}
```

This ensures that Jenkins works with the latest available source code.

9.2 Build Stage

The Build stage creates a Docker image using the Dockerfile.

```
stage('Build') {
    steps {
        sh '''
            docker build -t ecommerce-app:${BUILD_NUMBER} .
        '''
    }
}
```

The Jenkins build number is used as the image tag to distinguish between different builds.

9.3 Test Stage

The Test stage creates a temporary Docker container and checks whether the application responds correctly.

The test container runs on port 8082.

```
docker run -d --name ecommerce-test \
    -p 8082:80 ecommerce-app:<BUILD_NUMBER>
```

The application is tested using curl:

```
curl -fsS http://localhost:8082/ | grep -qi '<html'
```

If the application returns an unsuccessful HTTP response or the expected HTML check fails, the pipeline stops before deployment.

The temporary testing container is removed after execution.

9.4 Deploy Stage

After successful testing, Jenkins deploys the application.

The previous application container is removed, and a new container is started.

```
docker rm -f ecommerce-container


docker run -d \
    --name ecommerce-container \
    --restart unless-stopped \
```

-p 80:80 ecommerce-app:<BUILD_NUMBER>

The application is then verified through an HTTP request to port 80.

The website becomes accessible using the EC2 public IP address.

9.5 Pipeline Execution Flow

The complete pipeline follows this sequence:

1. Checkout – Retrieve source code.
2. Build – Generate Docker image.
3. Test – Verify application response.
4. Deploy – Run the updated application container.

The Deploy stage is executed only when the previous stages complete successfully.

10. Automatic Pipeline Triggering

Automatic triggering was implemented using Jenkins Poll SCM.

Poll SCM periodically checks the configured GitHub repository for new commits.

The following schedule was configured:

H/5 * * * *

This instructs Jenkins to check for changes approximately every five minutes.

When a new commit is detected, Jenkins automatically triggers the pipeline.

Demonstration

To verify automatic triggering:

1. A small modification was made to the application source code.
2. The updated code was committed and pushed to GitHub.
3. Jenkins detected the new commit during its polling cycle.
4. A new pipeline build was automatically initiated.
5. All four pipeline stages executed successfully.

This demonstrated that the pipeline could respond automatically to source code changes without manually selecting Build Now.

11. Testing and Results

The implementation was tested through both manual and automatic pipeline execution.

11.1 Functional Testing

Test Case	Expected Result	Observed Result
Terraform initialization	Provider initialized	Successful
Terraform validation	Configuration valid	Successful

Infrastructure provisioning	EC2 and security group created	Successful
Jenkins installation	Jenkins service running	Successful
Docker installation	Docker service running	Successful
GitHub integration	Source code retrieved	Successful
Docker image build	Image created	Successful
Application testing	Valid HTTP/HTML response	Successful
Application deployment	Container running on port 80	Successful
Manual pipeline execution	Pipeline completed	SUCCESS
Automatic triggering	New commit triggers pipeline	Verified

11.2 Deployment Result

The application was successfully deployed on the AWS EC2 instance.

The website was accessible through:

http://<EC2-PUBLIC-IP>

The Jenkins dashboard was accessible through:

http://<EC2-PUBLIC-IP>:8080

11.3 Pipeline Execution Result

The Jenkins console output displayed:

Application test passed!

Deployment successful!

CI/CD Pipeline completed successfully!

Finished: SUCCESS

The successful execution confirmed that the implemented pipeline completed the Checkout, Build, Test and Deploy stages.

12. Security Considerations

Security was considered during the implementation.

1. SSH access was restricted using a specific CIDR address.
2. Jenkins dashboard access was restricted to the configured public IP.
3. HTTP access was publicly available for website users.
4. The private EC2 key file was excluded from GitHub.
5. Terraform state files and environment-specific variable files were excluded from version control.
6. AWS credentials were not included in the source code repository.

7. A .gitignore file was maintained to prevent sensitive and unnecessary files from being committed.

13. Challenges and Solutions

During implementation, a few practical configuration challenges were encountered.

1. Jenkins and Java compatibility

Jenkins required a compatible Java runtime.

Solution: Java 21 was installed and verified before completing Jenkins installation.

2. Docker permission issues

Jenkins initially required appropriate permissions to execute Docker commands.

Solution: The Jenkins user was added to the Docker group and the Jenkins service was restarted.

3. Jenkins server connectivity

Access to the Jenkins dashboard depended on the EC2 instance state and security group configuration.

Solution: EC2 instance status, public IP address and inbound security group rules were checked.

4. Automatic deployment configuration

The pipeline needed to detect new source code changes without manual execution.

Solution: Jenkins Poll SCM was configured with a five-minute polling schedule.

14. Advantages of the Implemented System

- Reduces manual application deployment effort.
- Automates application building and testing.
- Provides consistent deployment using Docker.
- Enables repeatable infrastructure provisioning using Terraform.
- Integrates source code management with deployment automation.
- Detects source code changes automatically.
- Provides a cloud-based environment for hosting applications.
- Makes the deployment process easier to maintain and reproduce.

15. Limitations and Future Enhancements

Although the implementation successfully demonstrates a basic CI/CD workflow, it can be extended further.

Possible future enhancements include:

1. Implementing GitHub Webhooks instead of periodic polling.
2. Adding more comprehensive unit and integration tests.
3. Using a managed container registry such as Amazon ECR.
4. Separating Jenkins and application hosting into different EC2 instances.

5. Implementing HTTPS using SSL/TLS certificates.
6. Integrating monitoring and logging tools.
7. Using AWS IAM roles instead of manually configured credentials.
8. Implementing automated rollback in case of deployment failure.
9. Using Jenkins credentials management for secure deployment secrets.
10. Introducing Docker Compose or Kubernetes for more complex application deployments.

16. Conclusion

The project successfully demonstrates the implementation of an automated CI/CD pipeline using Jenkins, Docker, Terraform and AWS.

Terraform was used to provision the required AWS infrastructure, while Jenkins automated the application delivery workflow. Docker provided containerization, allowing the application to run in an isolated and consistent environment.

The pipeline successfully performed source code checkout, Docker image building, application testing and deployment. Automatic triggering was also verified through Jenkins Poll SCM by pushing updated code to GitHub.

The deployed e-commerce application was successfully accessed through the public IP address of the AWS EC2 instance.

Overall, this project provided practical experience in Infrastructure as Code, cloud computing, containerization, version control and CI/CD automation.

It demonstrates how multiple DevOps technologies can be integrated to create a reliable and automated application deployment workflow.

17. References

1. Jenkins Documentation – https://www.jenkins.io/doc/
2. Docker Documentation – https://docs.docker.com/
3. Terraform Documentation – https://developer.hashicorp.com/terraform/docs
4. AWS EC2 Documentation – https://docs.aws.amazon.com/ec2/
5. Git Documentation – https://git-scm.com/doc
6. GitHub Documentation – https://docs.github.com/